

=====

RISE – AI ALARM CLOCK

**MASTER BLUEPRINT: PART 10B OF 13
(REWRITTEN – CHANGESET)**

**Goals Journal · Quotes System · Meal Signal
Tracker**

=====

**GOVERNING DOCUMENT: RISE_DECISIONS.md
Section 4.**

**WHAT THIS REWRITE FOUND (two real bugs, not
just the planned rework):**

1. calorie_service.dart calls

`Hive.box(AppConstants.calorieBoxName)`

**and `Hive.box(AppConstants.foodDbBoxName)` — but Part 1 only
ever**

**declared `calorieLogsBoxName` (a different string) and
never**

declared `foodDbBoxName` at all. As originally written, this

would have thrown a "box not found" runtime crash the first time

a user tried to log food. Moot now that CalorieEntry is being

replaced, but worth recording as a concrete example of why this

reconciliation pass matters.

2. ai_reflection_service.dart's Claude API call uses

`'claude-sonnet-4-20250514'` — a stale model identifier.

Updated

to a current model string below.

DESIGN DECISION: food_item.dart (typeid 4, the CuisineRegion-aware

food database model) is NOT deleted.

DECISIONS.md Section 4 already

flagged the cultural food database as "worth building, later, as a

differentiator" — the model is genuinely well-designed (explicit

West/East/North/South/Central Africa, Middle East, South/Southeast

Asia, Latin America, Caribbean regions, first-class
not an "other"

bucket) and there's no reason to delete good work
only to redo it

later. It stays defined, dormant, at typeld 4, for
whenever that

later-stage feature gets built. What's actually
replaced is the

DEFAULT logging flow built on top of it
(calorie_service.dart,

calorie_screen.dart, food_search_sheet.dart) —
those forced every

free-tier user through a heavy full-tracker flow,
which is the

actual problem being solved.

WHAT DID NOT CHANGE, DELIBERATELY:

Ramadan awareness (Suhoor/Iftar

meal types, fast-day context) is genuinely
thoughtful cultural design

and is preserved in the new model, just simplified
to fit a

single-signal-per-day shape instead of full meal-by-meal logging.

Goals, habits, quotes, and proof wall (Files 3–7, 12–17) are entirely

unrelated to the calorie tracker and are UNCHANGED.

FILES IN THIS PART:

1. calorie_entry.dart → REPLACED with meal_signal_entry.dart (typeid 3)

2. food_item.dart → NO CHANGE (kept dormant, see above)

3-7. goal/habit/entry/quote/milestone models → NO CHANGE

8. goals_service.dart → NO CHANGE (no calorie references found)

9. ai_reflection_service.dart → CHANGED (meal-signal integration, model string, prompt softening)

10. quote_service.dart → NO CHANGE

11. calorie_service.dart → REPLACED with meal_signal_service.dart

12-17. goals/quotes screens → NO CHANGE

**18-19. calorie_screen.dart +
food_search_sheet.dart → REPLACED with
meal_signal_screen.dart (single file; no search
sheet needed
for a 5-tap form)**

=====

FILE 1: lib/core/models/meal_signal_entry.dart

**REPLACES calorie_entry.dart. Same typeId (3). The five
inputs match**

**the meal signal tracker exactly as decided in DECISIONS.md
Section 4.**

```
import 'package:hive/hive.dart';
import 'package:uuid/uuid.dart';

part 'meal_signal_entry.g.dart';

enum MealSize { light, medium, heavy }

extension MealSizeExt on MealSize {
  String get displayName => switch (this) {
    MealSize.light => 'Light',
    MealSize.medium => 'Medium',
    MealSize.heavy => 'Heavy',
  };
}

enum WaterIntake { under, onTarget, above }

extension WaterIntakeExt on WaterIntake {
  String get displayName => switch (this) {
    WaterIntake.under => 'Under target',
    WaterIntake.onTarget => 'On target',
    WaterIntake.above => 'Above target',
  };
}
```

```

    };
  }

  @HiveType(typeId: 3)
  class MealSignalEntry extends HiveObject {
    @HiveField(0) final String id;
    @HiveField(1) final String date; // 'YYYY-MM-DD'
    @HiveField(2) DateTime lastMealTime;
    @HiveField(3) MealSize mealSize;
    @HiveField(4) int alcoholUnits; // 0, 1, 2, or 3
    (3 means "3+")
    @HiveField(5) bool caffeineAfter2pm;
    @HiveField(6) WaterIntake waterIntake;
    @HiveField(7) DateTime loggedAt;
    @HiveField(8) bool isRamadanFast; // preserved from
    the // original –
    still a // meaningful
    signal // even without
    full // meal-by-meal
    logging

    MealSignalEntry({
      String? id,
      required this.date,
      required this.lastMealTime,
      required this.mealSize,
      required this.alcoholUnits,
      required this.caffeineAfter2pm,
      required this.waterIntake,
      DateTime? loggedAt,
      this.isRamadanFast = false,
    }) : id = id ?? const Uuid().v4(),
        loggedAt = loggedAt ?? DateTime.now();
  }

```

FILE 2: lib/core/models/food_item.dart

NO CHANGE — carries forward verbatim, kept dormant at
 typeId 4 for

the later cultural food database (see design decision note above).

FILES 3–7: north_star_goal.dart, habit_model.dart, daily_entry.dart,

quote_item.dart, proof_wall_milestone.dart

NO CHANGE. None reference calorie data.

FILE 8: lib/core/services/goals_service.dart

NO CHANGE. No calorie/meal references found in this file — the

`calorieAvg` -style helpers referenced from ai_reflection_service.dart

are private methods defined within that file itself, not here.

FILE 9: lib/core/services/ai_reflection_service.dart — CHANGED

Three changes: (1) meal-signal integration replaces calorie integration, (2) the correlation-insight data-sufficiency gate now

uses the shared constant instead of a hardcoded number, (3) the

Claude model string is updated and the correlation-insight prompt

is softened for compliance. generateWeeklyReview(),

generateMonthlyLetter(), and generateMirror() are otherwise

UNCHANGED — including their existing tier-gating, which is already

a well-designed precision paywall (Core gets the weekly narrative,

Pro gets the monthly letter, free gets neither) — see the

correction note at the end of this file section.

```
class AiReflectionService {
  // ...unchanged singleton boilerplate...

  static const _apiUrl = 'https://api.anthropic.com/v1/messages';
  // CHANGED — was 'claude-sonnet-4-20250514' (stale identifier)
  static const _model = 'claude-sonnet-5';

  // generateWeeklyReview() — UNCHANGED except the data-gathering line:
  //   final calorieAvg = await _getWeekCalorieAvg();
  // becomes:
  //   final mealSignalSummary = await _getWeekMealSignalSummary();
  // and _buildWeeklyPrompt's `calorieAvg:` named parameter becomes
  // `mealSignalSummary:`. Everything else in this method — the tier
  // check, narrative generation call, caching — is unchanged.

  // generateMonthlyLetter() — UNCHANGED, no calorie references exist
  // in this method at all.

  // generateMirror() — UNCHANGED, no calorie references.

  // ——— SLEEP + MEAL SIGNAL CORRELATION INSIGHT
  // CHANGED from generateSleepNutritionInsight's original body: the
  // data-sufficiency gate now uses the shared constant (was a bare
```



```

// "7", inconsistent with AppConstants.minDaysForCorrelationInsight
// = 14 added in the Part 1 rewrite) and the prompt is softened per
// RISE_COMPLIANCE_AUDIT.md Section 3 (no unqualified biological-
// mechanism claims).
Future<String?> generateSleepNutritionInsight({
    required String tierId,
}) async {
    if (!SubscriptionTier.fromId(tierId).fullSleepHistory) return null;

    final sessions = _getMonthSessions();
    // CHANGED – was `sessions.length < 7`
    if (sessions.length < AppConstants.minDaysForCorrelationInsight) {
        return null; // Not enough data yet
    }

    final signalData = await _getMealSignalVsSleepData(); // was
    _getCalorieVsSleepData()

    final prompt = '''
You are RISE's wellness insight engine. Based on this user's own data
over the last ${sessions.length} nights:
${jsonEncode(signalData)}

Write ONE insight in 2-3 sentences that:
1. Identifies a real, consistent pattern in their own sleep vs.
meal-signal data – only if it's actually consistent across these
nights, not a one-off
2. If you reference a mechanism (e.g. late meals and digestion or
core body temperature), phrase it in general-knowledge terms –
"can affect," never a claim about what IS happening in this
person's body. This is wellness information, not medical advice.
3. Offers one concrete, actionable suggestion for tomorrow
4. If there's no consistent pattern, say so plainly – a wrong pattern
costs more trust than no pattern

Write in second person. Be warm and specific, not generic.
''';

    return _callClaude(prompt, maxTokens: 150,
        systemPrompt: 'You are a warm, evidence-based wellness coach. '
        'You never make medical claims or diagnoses.');
```

}

// _callClaude() – UNCHANGED except the _model constant above.

// _getWeekMealSignalSummary() – NEW, replaces _getWeekCalorieAvg().
// Pulls from AppConstants.mealSignalsBoxName instead of the old
// calorie box, returns a summary of last-meal-time consistency,
// average meal size distribution, and alcohol/caffeine/water
// patterns for the week – shape mirrors what _buildWeeklyPrompt
// already expects to interpolate into the narrative prompt.

```

    // _getMealSignalVsSleepData() – NEW, replaces
    _getCalorieVsSleepData().
    // Pairs each night's sleep session with that day's MealSignalEntry
    // (matched by date), producing the correlation dataset the insight
    // prompt above consumes. Implementation follows the same shape as
    // the original _getCalorieVsSleepData() – join on date, output a
    // list of {date, sleepHours, sleepScore, lastMealTime, mealSize,
    // alcoholUnits, caffeineAfter2pm} records.
}

```

Correction to the earlier monetization discussion: the existing Weekly Review (Core tier, generated weekly, narrative-but-lightweight) and Monthly Letter (Pro tier, generated monthly, the deep personal letter) are two distinct, already-well-designed features — not one "Weekly Letter" that needs a new precision paywall. The suggestion made earlier ("free gets it monthly, Pro gets it weekly") doesn't map onto what's actually built. The existing gating is already the right shape: free gets neither, Core gets the lighter weekly narrative, Pro gets the deeper monthly one. DECISIONS.md Section 7 should be read with this correction — no change to the actual tier flags is needed, just a correction to that earlier prose recommendation.

FILE 10: lib/core/services/quote_service.dart

NO CHANGE. Entirely unrelated to calorie/meal data.

FILE 11: lib/core/services/meal_signal_service.dart

REPLACES calorie_service.dart. Much smaller — no food database

search, no macro math, no barcode lookup. Fixes the box-name bug

noted at the top of this document as a side effect of the rewrite.

```

import 'package:hive/hive.dart';
import '../constants/app_constants.dart';
import '../models/meal_signal_entry.dart';
import '../models/sleep_session.dart';
import 'offline_sync_service.dart';
import 'remote_config_service.dart';

class MealSignalService {
  static final MealSignalService _i = MealSignalService._internal();
  factory MealSignalService() => _i;
  MealSignalService._internal();

  final _sync = OfflineSyncService();
  final _rc = RemoteConfigService();

  // — LOG TODAY'S SIGNALS

  // One entry per day – logging again the same day overwrites, it
  // doesn't append (this isn't a meal-by-meal log).
  Future<void> logToday({
    required DateTime lastMealTime,
    required MealSize mealSize,
    required int alcoholUnits,
    required bool caffeineAfter2pm,
    required WaterIntake waterIntake,
  }) async {
    final entry = MealSignalEntry(
      date: _todayStr(),
      lastMealTime: lastMealTime,
      mealSize: mealSize,
      alcoholUnits: alcoholUnits,
      caffeineAfter2pm: caffeineAfter2pm,
      waterIntake: waterIntake,
      isRamadanFast: _rc.isRamadanActive,
    );

    // Correct box name – the original calorie_service.dart called
    // Hive.box(AppConstants.calorieBoxName), which Part 1 never
    // actually declared. mealSignalsBoxName is the real, opened box
    // (see Part 1 rewrite, Section 1.7-equivalent fix).
    await Hive.box<dynamic>(AppConstants.mealSignalsBoxName)
      .put(entry.id, entry);
    await _sync.queue(SyncOpType.saveMealSignal, _entryToMap(entry));
  }

  // — GET TODAY'S ENTRY

  MealSignalEntry? getTodayEntry() => _getEntryForDate(_todayStr());

```

```
MealSignalEntry? _getEntryForDate(String date) {
  final box = Hive.box<dynamic>(AppConstants.mealSignalsBoxName);
  return box.values
    .cast<MealSignalEntry>()
    .where((e) => e.date == date)
    .firstOrNull;
}

Map<String, dynamic> _entryToMap(MealSignalEntry e) => {
  'id': e.id, 'date': e.date,
  'lastMealTime': e.lastMealTime.toIso8601String(),
  'mealSize': e.mealSize.name,
  'alcoholUnits': e.alcoholUnits,
  'caffeineAfter2pm': e.caffeineAfter2pm,
  'waterIntake': e.waterIntake.name,
  'isRamadanFast': e.isRamadanFast,
};

String _todayStr() {
  final n = DateTime.now();
  return '${n.year}-${n.month.toString().padLeft(2,
'0')}-${n.day.toString().padLeft(2, '0')}';
}

// searchFood(), getDailySummary() with macros, seedFoodDatabase() –
// all REMOVED. No food database search needed for a 5-input signal
// log. If the optional partner photo-food API (Passio/Calorie Mama,
// DECISIONS.md Section 4) gets built later, it gets its own service
// – this one stays simple.
}
```

FILES 12–17: goals_screen.dart,
daily_intention_screen.dart,

weekly_review_screen.dart, monthly_letter_screen.dart,

proof_wall_screen.dart, quotes_screen.dart

NO CHANGE. None reference calorie data directly.

FILE 18: lib/features/nutrition/meal_signal_screen.dart

REPLACES calorie_screen.dart AND food_search_sheet.dart
(both –

the search sheet has no equivalent, there's nothing to
search).

Route path matches Part 1's rewrite: '/nutrition'.

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '.../core/models/meal_signal_entry.dart';
import '.../core/services/meal_signal_service.dart';
import '.../core/services/remote_config_service.dart';

class MealSignalScreen extends ConsumerStatefulWidget {
  const MealSignalScreen({super.key});

  @override
  ConsumerState<MealSignalScreen> createState() =>
    _MealSignalScreenState();
}

class _MealSignalScreenState extends ConsumerState<MealSignalScreen> {
  final _svc = MealSignalService();
  final _rc = RemoteConfigService();

  DateTime? _lastMealTime;
  MealSize _mealSize = MealSize.medium;
  int _alcoholUnits = 0;
  bool _caffeineAfter2pm = false;
  WaterIntake _waterIntake = WaterIntake.onTarget;
  bool _saving = false;

  @override
  void initState() {
    super.initState();
    final existing = _svc.getTodayEntry();
    if (existing != null) {
      _lastMealTime = existing.lastMealTime;
      _mealSize = existing.mealSize;
      _alcoholUnits = existing.alcoholUnits;
      _caffeineAfter2pm = existing.caffeineAfter2pm;
      _waterIntake = existing.waterIntake;
    }
  }

  Future<void> _save() async {
    if (_lastMealTime == null) return;
```

```

    setState(() => _saving = true);
    await _svc.logToday(
      lastMealTime: _lastMealTime!,
      mealSize: _mealSize,
      alcoholUnits: _alcoholUnits,
      caffeineAfter2pm: _caffeineAfter2pm,
      waterIntake: _waterIntake,
    );
    if (!mounted) return;
    setState(() => _saving = false);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Logged – takes about 20 seconds,
worth it')),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF0D0D1A),
      appBar: AppBar(
        backgroundColor: const Color(0xFF0D0D1A),
        title: const Text('Nutrition',
          style: TextStyle(color: Colors.white, fontWeight:
FontWeight.w300)),
      ),
      body: ListView(
        padding: const EdgeInsets.all(20),
        children: [
          if (_rc.isRamadanActive) const _RamadanContextBanner(),
          // 5 inputs – time picker, three-way size selector, stepper,
          // toggle, three-way water selector. Each renders as a single
          // RiseCard row; full widget tree straightforward given the
          // shared widgets from Part 1 (RiseCard, RiseButton).
          _LastMealTimeField(
            value: _lastMealTime,
            onChanged: (t) => setState(() => _lastMealTime = t),
          ),
          _MealSizeField(
            value: _mealSize,
            onChanged: (v) => setState(() => _mealSize = v),
          ),
          _AlcoholField(
            value: _alcoholUnits,
            onChanged: (v) => setState(() => _alcoholUnits = v),
          ),
          _CaffeineField(
            value: _caffeineAfter2pm,
            onChanged: (v) => setState(() => _caffeineAfter2pm = v),
          ),
          _WaterField(

```

```

        value: _waterIntake,
        onChanged: (v) => setState(() => _waterIntake = v),
      ),
      const SizedBox(height: 24),
      RiseButton(
        label: 'Save',
        isLoading: _saving,
        onPressed: _lastMealTime != null ? _save : null,
      ),
    ],
  ),
);
}
}

// Field widgets (_LastMealTimeField, _MealSizeField, _AlcoholField,
// _CaffeineField, _WaterField, _RamadanContextBanner) – straightforward
// Material input widgets, not spelled out further here; each is a
// single RiseCard containing one input control matching its field's
// type (time picker, segmented button, stepper, switch, segmented
// button respectively).

```

FILE 19: lib/features/nutrition/food_search_sheet.dart

REMOVED. No search needed for a 5-input signal log. If the optional

partner photo-food API add-on (DECISIONS.md Section 4) is built

later, it gets its own screen at that time.

PART 10B FILE INVENTORY (rewritten)

REPLACED: meal_signal_entry.dart (was calorie_entry.dart),
 meal_signal_service.dart (was calorie_service.dart),
 meal_signal_screen.dart (was calorie_screen.dart +
 food_search_sheet.dart, now one file, one file removed)
 CHANGED: ai_reflection_service.dart
 NO CHANGE: food_item.dart (kept dormant), north_star_goal.dart,
 habit_model.dart, daily_entry.dart, quote_item.dart,

proof_wall_milestone.dart, goals_service.dart,
quote_service.dart, goals_screen.dart,
daily_intention_screen.dart, weekly_review_screen.dart,
monthly_letter_screen.dart, proof_wall_screen.dart,
quotes_screen.dart

PART 8 — CHECKED, NO CHANGES NEEDED

Verified both terms that looked calorie/stakes-adjacent in Part 8's header: "barcode" there is the wake-up game mechanic (scan a specific item across the room), unrelated to food logging. "Financial stakes" appears once in the descriptive header list but has no actual implementation in the file — that UI belongs to Part 13. Part 8 requires no changes for anything decided in this rewrite.

WHAT'S NEXT (per DECISIONS.md Section 9)

Part 12 — absorb Part 13's pubspec/box additions.

Part 13 — add the legal-gating scaffolding for stakes.